Logic Minimization – Part 4

K. Sridharan

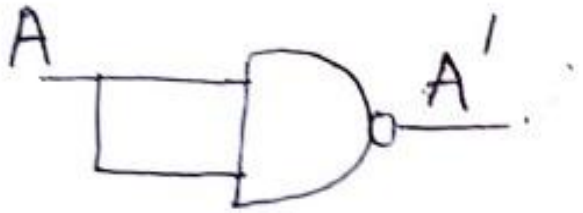
Review

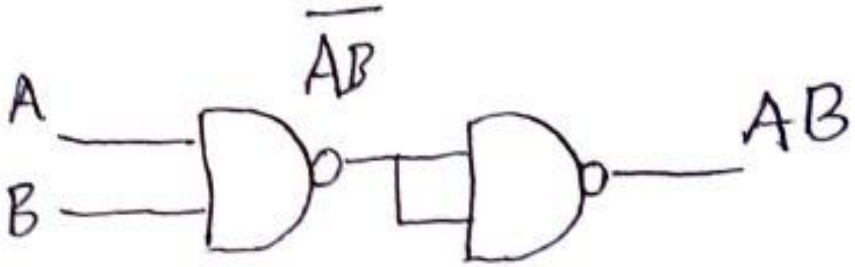
- We have discussed essential prime implicants.
- We will now look at NAND and NOR implementation of various functions.

Recall

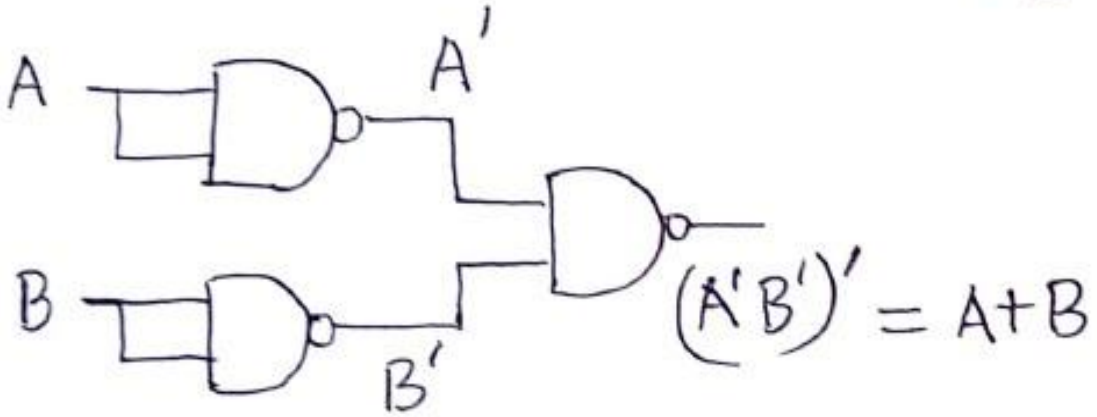
- NAND and NOR are universal gates. They can be therefore used (solely) to implement various Boolean functions.
- We illustrate first how the basic functions, namely NOT (inverter), AND and OR are realized using NAND and NOR.
- Recall DeMorgan's Laws too for seeing the truth of the realizations.

NAND implementation of NOT, AND and OR

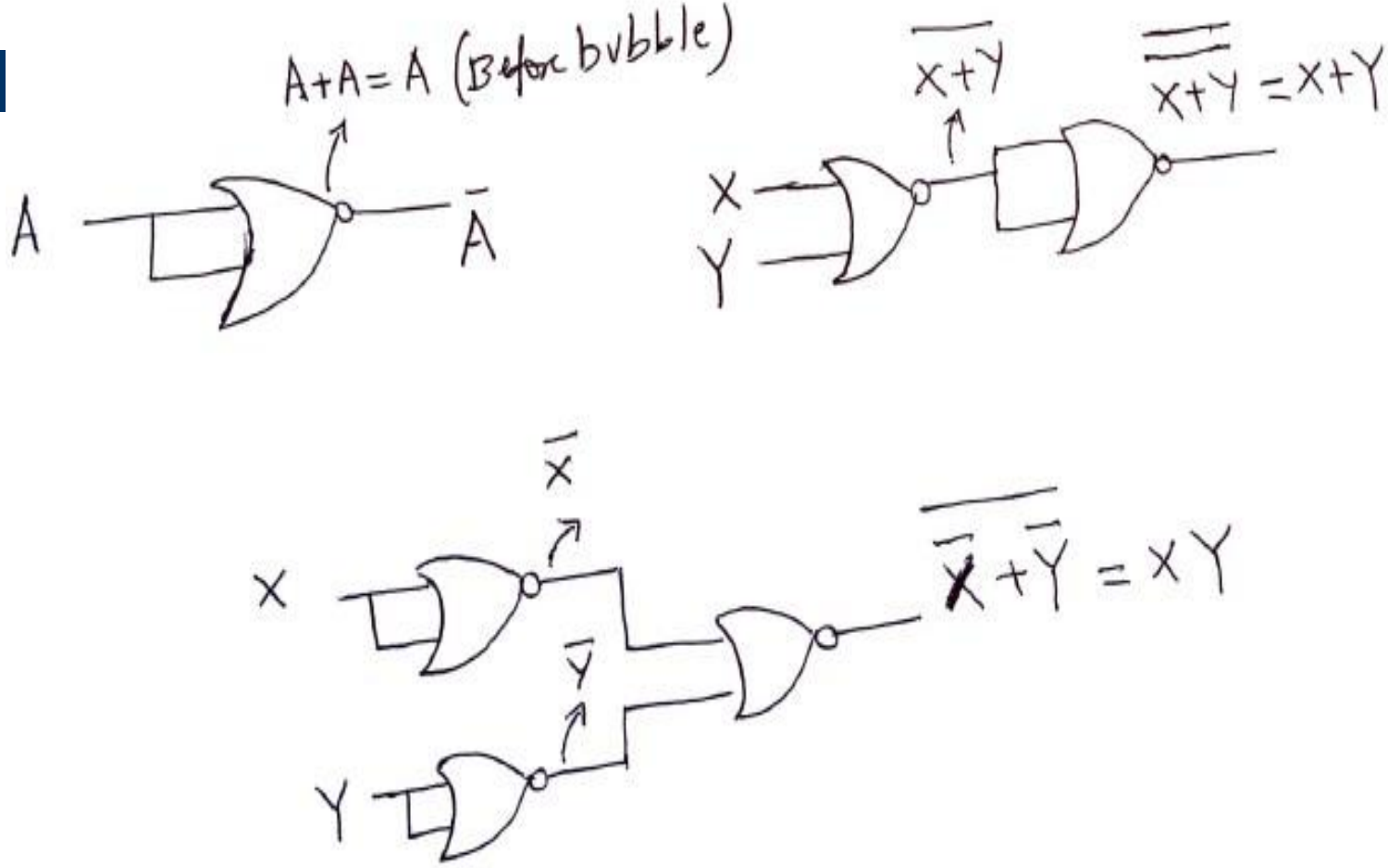




$\overline{AB}$ is same as $(AB)'$



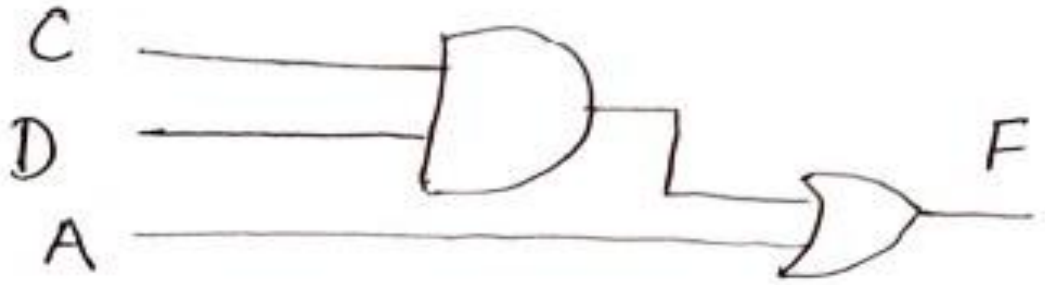
NOR implementation



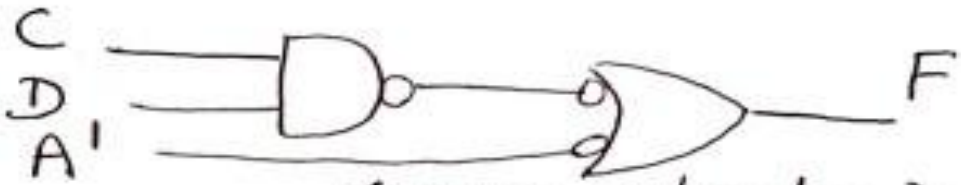
How to design a minimum NAND-NAND network for a function F ?

- Find a minimum sum-of-products expression for F (using K-Map).
- Draw the corresponding two-level AND-OR network.
- Replace all gates with NAND gates leaving gate interconnections unchanged. If the output gate has any single literals as input, complement these literals. Example next.

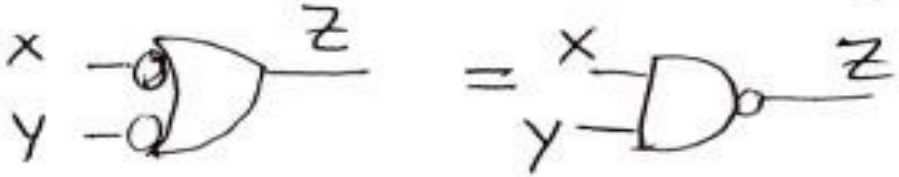
Example



(AND-OR realization)



(NAND realization)

Note 

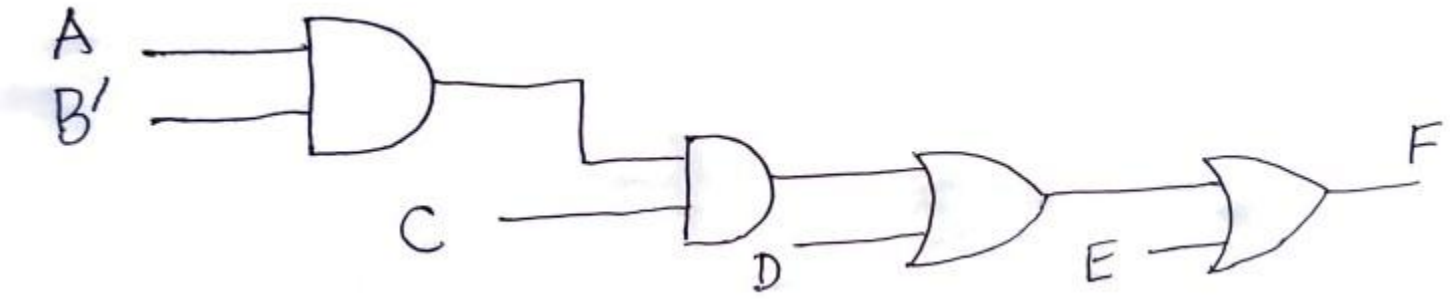
(by DeMorgan's Law)

What about NOR-NOR realization ?

- We need a minimum product-of-sums expression for the function F.
- Draw the corresponding OR-AND network.
- Then replace all gates with NOR gates leaving the gate interconnections unchanged. If the output gate has any single literals as inputs, complement these literals.

What happens if you have a network with more than 2 levels ?

Further, suppose that we do not have AND followed by OR throughout (i.e., AND and OR do not alternate). As an example, consider: F = [C(AB') + D] + E. The AND-OR network is:

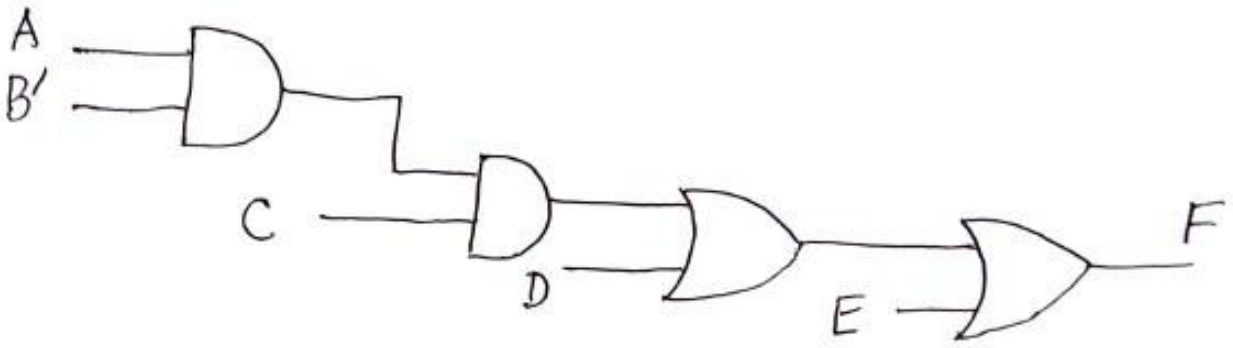


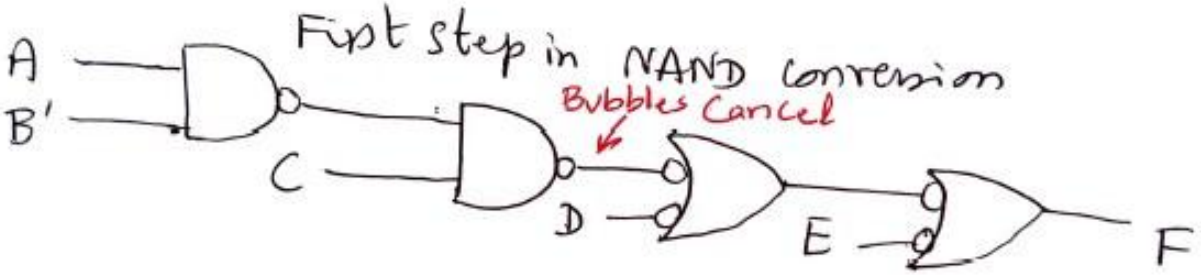
How to convert systematically into a NAND (only) network ?

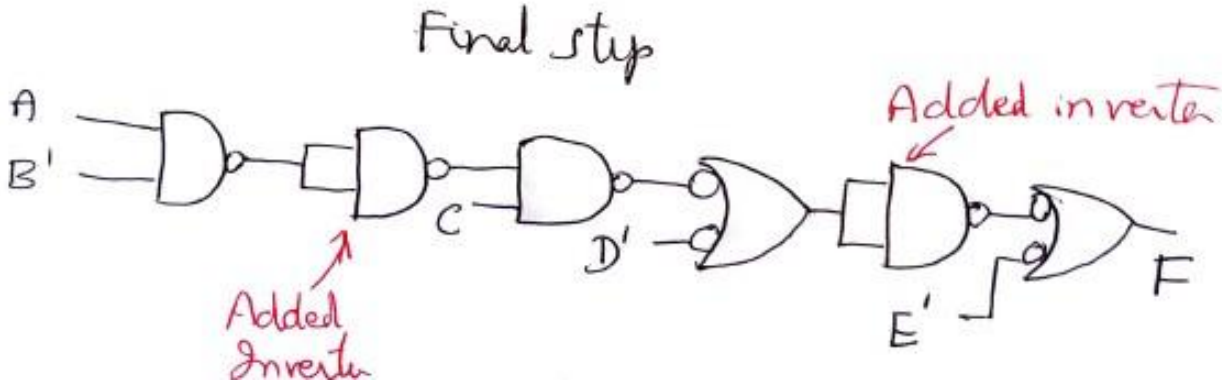
- Convert all AND gates to NAND gates by adding an inversion bubble at the output. Convert all OR gates to NAND gates by adding inversion bubbles at the inputs.
- If a noninverted gate output drives an inverted gate input (or vice-versa), insert an inverter so that the bubbles will cancel (note that if an inverted output drives an inverted input, no extra work is required).
- When a variable drives an inverted input, complement the variable (or add an inverter in NAND form). This way, the complementation cancels the inversion at the input.

Illustration

F = [C(AB') + D] + E







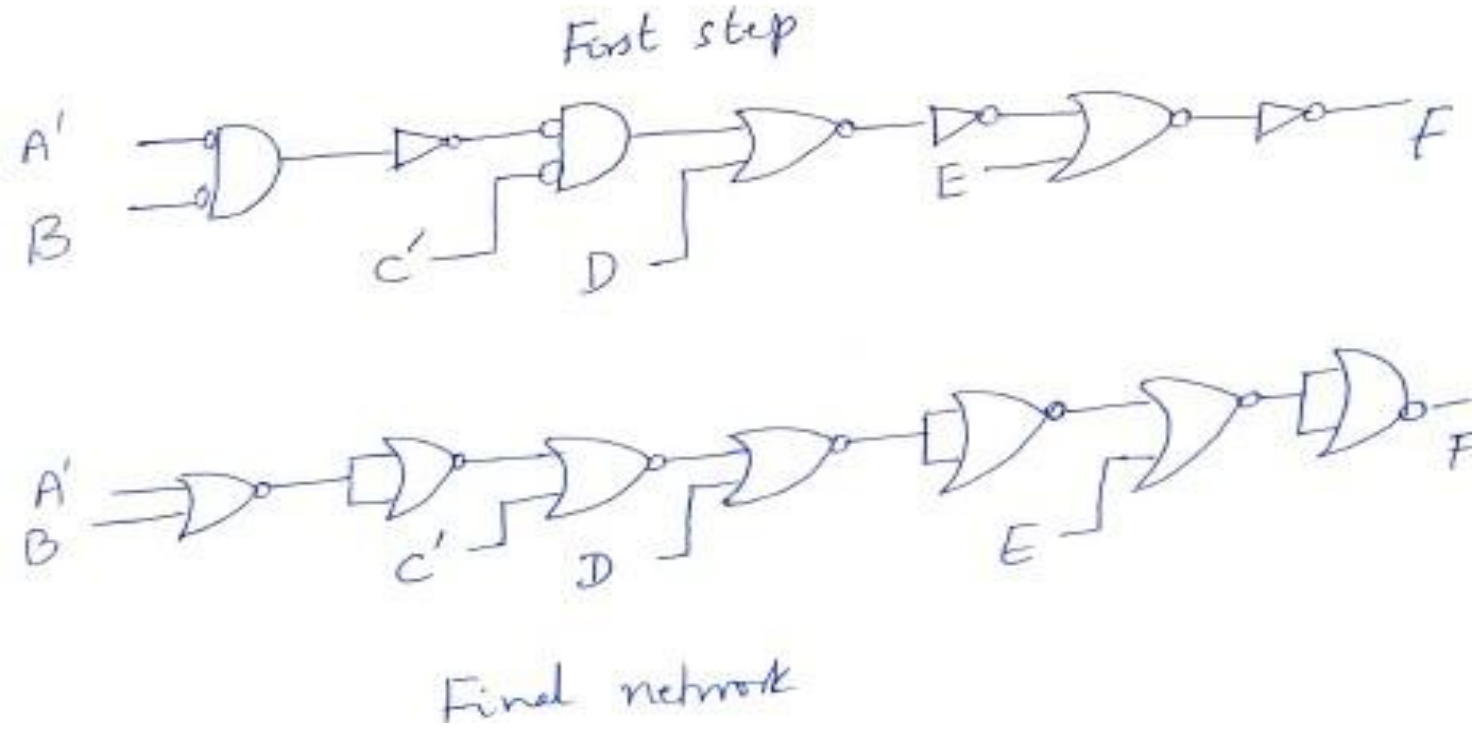
How to convert systematically into a NOR (only) network ?

- Convert all AND and OR gates to NOR gates. Add inversion bubbles at all OR gate outputs. Also, add inversion bubbles at all AND gate inputs.
- If a noninverted gate output drives an inverted gate input (or vice-versa), insert an inverter so that the bubbles will cancel. Choose an inverter with the bubble at the input or output as required. Note that if an inverted output drives an inverted input, no extra work is required.
- When a variable drives an inverted input, complement the variable (or add an inverter in NOR form). This way, the complementation cancels the inversion at the input.

Illustration

F = [C(AB') + D] + E





Textbook and Reference

- M. Mano and M. Ciletti, Digital Design, Pearson Education, 2018
- C. Roth and L.L. Kinney, Fundamentals of Logic Design, Gamma Publications, 2021